Lecture 8
Constraint-based search
Continued

Armando Solar-Lezama

The inductive synthesis problem

$$\exists \phi \; \forall in \in E \; Q(in, \phi)$$

where E = $\{x_1, x_2, \ldots, x_k\}$

Overall Strategy



Building Constraints

Program sketching

Armando Solar-Lezama, International Journal on Software Tools for Technology Transfer
October 2013, Volume 15, Issue 5-6, pp 475-495

Semantics of a simple language

e:= n | x | e_1 + e_2 | e_1 > e_2
c:= x := e | c_1 ; c_2 | if e then c_1 else c_2 | while e do c

What does an expression mean?

- An expression reads the state and produces a value
- The state is modeled as a map σ from vars to values
- $\mathcal{A}[\![\cdot]\!] : e \to \Sigma \to int$

Ex:

- $\mathcal{A}[\![x]\!] = \lambda\sigma.\,\sigma(x)$
- $\mathcal{A}[\![n]\!] = \lambda\sigma.\,n$
- $\mathcal{A}[\![e_1 + e_2]\!] = \lambda\sigma.\,\mathcal{A}[\![e_1]\!]\sigma + \mathcal{A}[\![e_2]\!]\sigma$
- $\mathcal{A}[\![e_1 > e_2]\!] = \lambda\sigma.\,if\ \mathcal{A}[\![e_1]\!]\sigma > \mathcal{A}[\![e_2]\!]\sigma\ then\ 1\ else\ 0$

Semantics of a simple language

e:= n | x | $e_1 + e_2$
c:= x := e | c_1 ; c_2 | if e then c_1 else c_2 | while e do c

What does a command mean?

- A command modifies the state
- $\mathcal{C}[\![\cdot]\!] : c \to \Sigma \to \Sigma$

Ex:

- $\mathcal{C}[\![x := e]\!] = \lambda\sigma.\sigma[x \to (\mathcal{A}[\![e]\!]\sigma)]$
- $\mathcal{C}[\![c_1; c_2]\!] = \lambda\sigma.\mathcal{C}[\![c_2\ ]\!](\mathcal{C}[\![c_1\ ]\!]\sigma)$
- $\mathcal{C}[\![if\ e\ then\ c_1 else\ c_2]\!] =$
  $\lambda\sigma.\lambda x.if\ \mathcal{A}[\![e]\!]\sigma = 1\ then\ (\mathcal{C}[\![c_1\ ]\!]\sigma)x\ else(\mathcal{C}[\![c_2\ ]\!]\sigma)x$

What about loops?

Semantics of a while loop

- Let $W = \mathcal{C}[\![while\ e\ do\ c]\!]$
- W satisfies the following equation:

$$W = \lambda\sigma.\lambda x.\, if\, \mathcal{A}[\![e]\!] then\, \big(W(\mathcal{C}[\![c\,]\!]\sigma)\big)x\ else\ \sigma x$$

- Equation can have many solutions
  - when loop doesn't terminate
- Rich theory for finding least fixed point solution
- We'll settle for a simpler strategy:

  unroll k times and then give up

Symbolic execution of sketches

Very similar to what we just saw

- But values are now parameterized by ϕ

$\Psi = \boldsymbol{\Phi} \to \mathbb{Z}$

$$\mathcal{A}[\![\circ]\!]^{\tau} : Aexp \to (\Sigma \to \Psi)$$

The denotation function will keep track of contexts

$$\mathcal{A}[\![x]\!]^{\tau}\sigma = \sigma(x)$$

$$\mathcal{A}[\![??_i]\!]^{\tau}\sigma = \lambda\phi.\phi(??_i, \tau)$$

$$\mathcal{A}[\![e_1 \star e_2]\!]^{\tau}\sigma = \lambda\phi.\ \mathcal{A}[\![e_1]\!]^{\tau}\sigma\phi \star \mathcal{A}[\![e_2]\!]^{\tau}\sigma\phi$$

Symbolic Evaluation of Commands

Commands have two roles

- Modify the symbolic state
- Generate constraints

$$\mathcal{C}[\![\circ]\!]^{\tau} : Command \rightarrow (\langle \Sigma, \mathcal{P}(\boldsymbol{\Phi}) \rangle \rightarrow \langle \Sigma, \mathcal{P}(\boldsymbol{\Phi}) \rangle)$$

Constraints represent sets of valid ϕ functions

Symbolic Evaluation of Commands

Assignments and Assertion

$$\mathcal{C}[\![x := e]\!]^\tau \langle \sigma , \Phi \rangle = \langle \sigma[x \mapsto \ \mathcal{A}[\![e]\!]^\tau \sigma] , \Phi \rangle$$

$$\mathcal{C}[\![\textbf{assert } e]\!]^\tau \langle \sigma , \Phi \rangle = \langle \sigma , \{\phi \in \Phi : \ \mathcal{A}[\![e]\!]^\tau \sigma \phi = 1\} \rangle$$

Symbolic Evaluation of Commands

If statement

$$\mathcal{C}[\![\textbf{if } e \textbf{ then } c_1 \textbf{ else } c_2]\!]^\tau \langle \sigma , \Phi \rangle = \langle \sigma' , \Phi' \rangle$$

$$\Phi_t = \{\phi \in \Phi : \ \mathcal{A}[\![e]\!]^\tau \sigma \phi = true\}$$
$$\Phi_f = \{\phi \in \Phi : \ \mathcal{A}[\![e]\!]^\tau \sigma \phi = false\}$$
$$\langle \sigma_1 , \Phi_1 \rangle = \ \mathcal{C}[\![c_1]\!]^\tau \langle \sigma , \Phi_t \rangle$$
$$\langle \sigma_2 , \Phi_2 \rangle = \ \mathcal{C}[\![c_2]\!]^\tau \langle \sigma , \Phi_f \rangle$$
$$\Phi' = (\Phi_1) \cup (\Phi_2)$$
$$\sigma' = \lambda x.\lambda \phi. \ \mathcal{A}[\![e]\!]^\tau \sigma \phi \ ? \ \sigma_1 x \phi \ : \ \sigma_2 x \phi$$

Conditionals

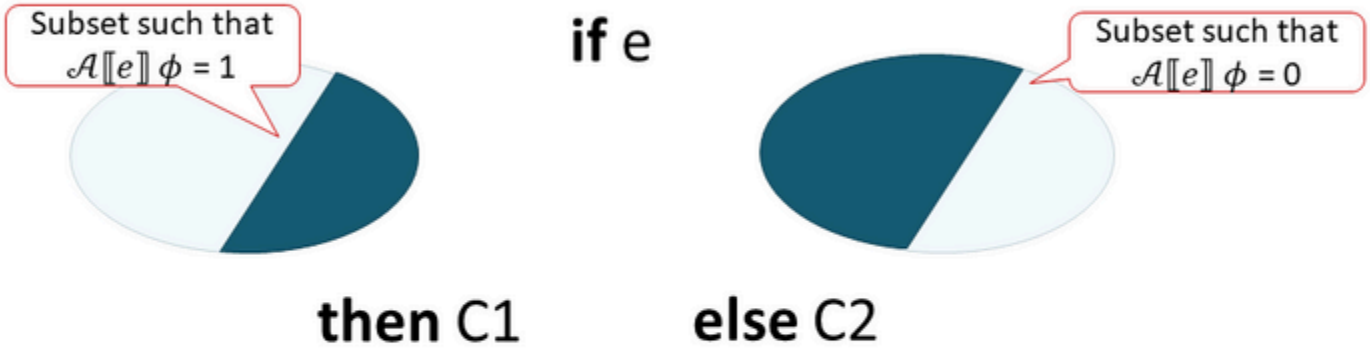


Initial set of viable ϕ

if e

then C1 **else** C2

Conditionals



Conditionals

if e

then C1



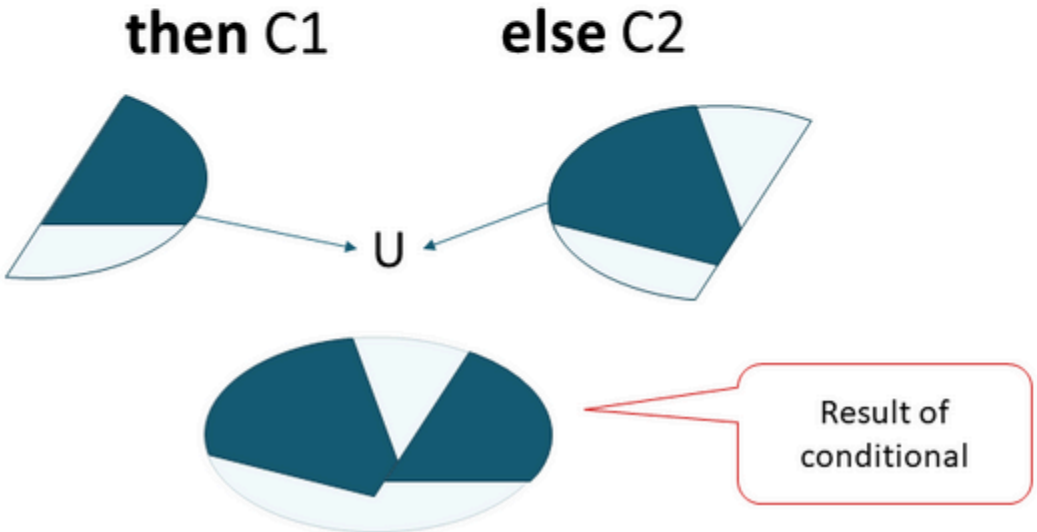
else C2



Subset that also passes all the assertions in C2

Conditionals

if e



Symbolic Evaluation of Commands

While loops

$$W(\langle \sigma\,,\,\Phi\rangle) = \mathcal{C}[\![\textbf{while } e \textbf{ do } c]\!]^{\tau}\langle \sigma\,,\,\Phi\rangle = \langle \sigma'\,,\,\Phi'\rangle$$

$$\Phi_t = \{\phi \in \Phi :\ \mathcal{A}[\![e]\!]^{\tau}\sigma\phi = true\}$$

$$\Phi_f = \{\phi \in \Phi :\ \mathcal{A}[\![e]\!]^{\tau}\sigma\phi = false\}$$

$$\langle \sigma_1\,,\,\Phi_1\rangle = W(\ \mathcal{C}[\![c]\!]^{\tau}\langle \sigma\,,\,\Phi_t\rangle)$$

$$\Phi' = (\Phi_1) \cup (\Phi_f)$$

$$\sigma' = \lambda x.\lambda\phi.\ \mathcal{A}[\![e]\!]^{\tau}\sigma\phi\ ?\ \sigma_1 x\phi\ :\ \sigma x\phi$$

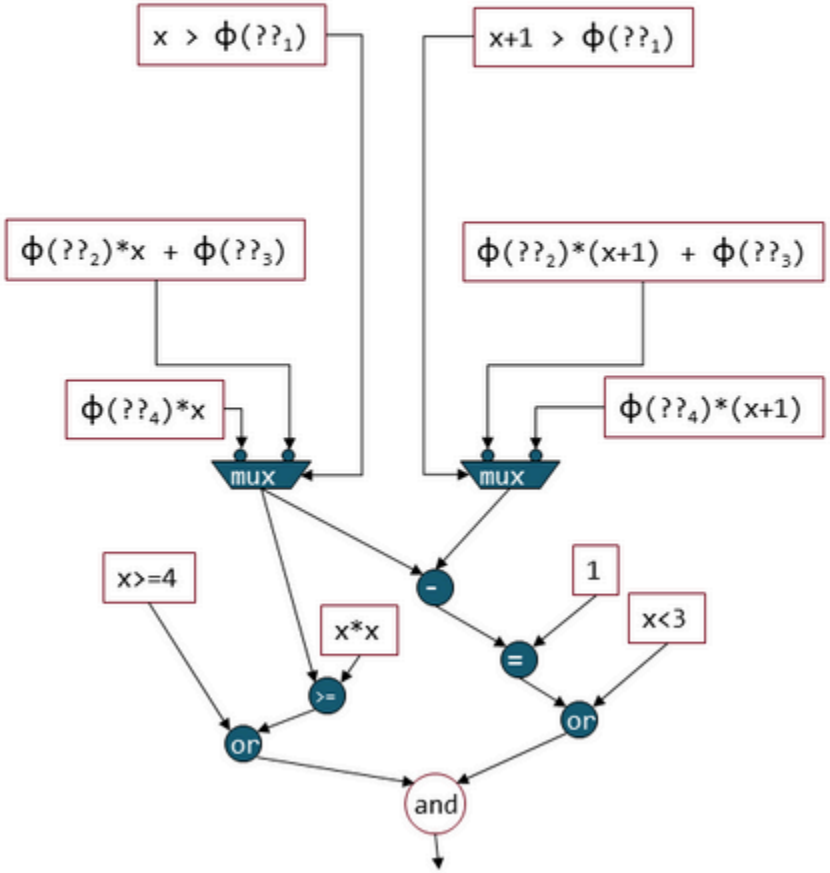
A sketch as a constraint system

```
int lin(int x){
   if(x > ϕ(??1))
      return ϕ(??2)*x + ϕ(??3);
   else
      return ϕ(??4)*x;
}

void main(int x){
   int t1 = lin(x);
   int t2 = lin(x+1);

   if(x<4) assert t1 >=  x*x;

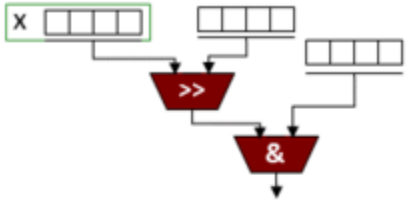
   if(x>=3) assert t2-t1 == 1;
}
```



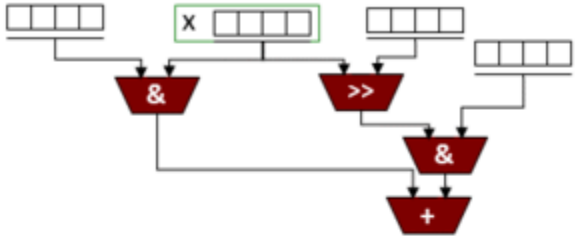
```
int popSketched (bit[W] x)
    implements pop {
    repeat(??){
        x = (x & ??)
        + ((x >> ??) & ??);
    }
    return x;
}
```




```
int popSketched (bit[W] x)
    implements pop {
    repeat(??){
        x = (x & ??)
        + ((x >> ??) & ??);
    }
    return x;
}
```

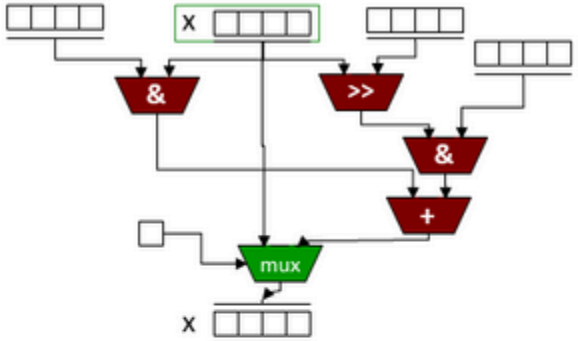



```
int popSketched (bit[W] x)
    implements pop {
    repeat(??){
        x = (x & ??)
        + ((x >> ??) & ??);
    }
    return x;
}
```

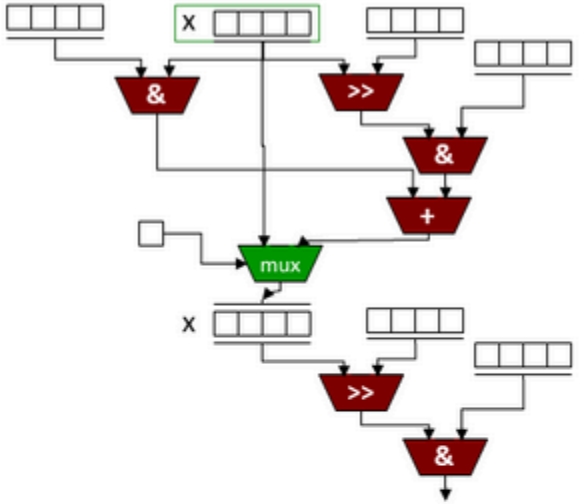



```
int popSketched (bit[W] x)
    implements pop {
    repeat(??) {
        x = (x & ??)
        + ((x >> ??) & ??);
    }
    return x;
}
```

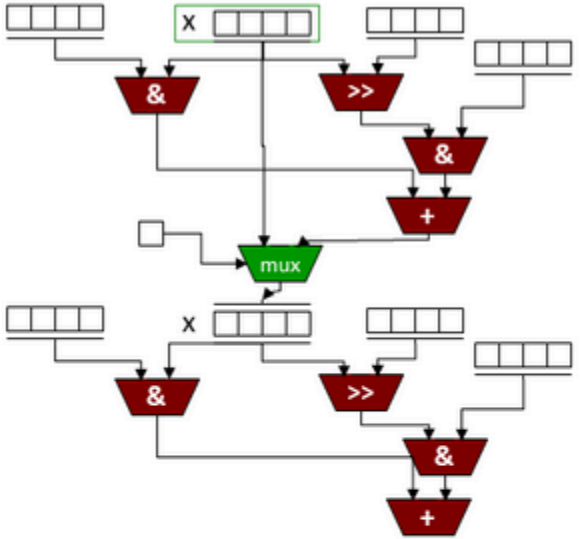
```
int popSketched (bit[W] x)
    implements pop {
    repeat(??) {
        x = (x & ??)
        + ((x >> ??) & ??);
    }
    return x;
}
```



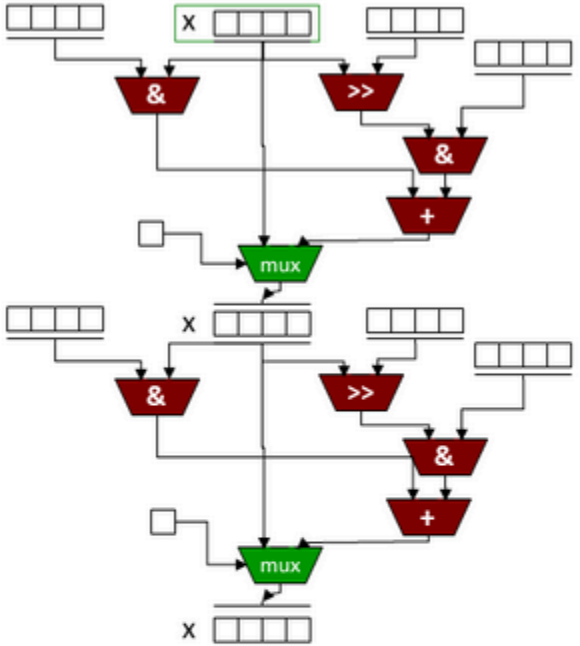
```
int popSketched (bit[W] x)
    implements pop {
    repeat(??) {
        x = (x & ??)
        + ((x >> ??) & ??);
    }
    return x;
}
```



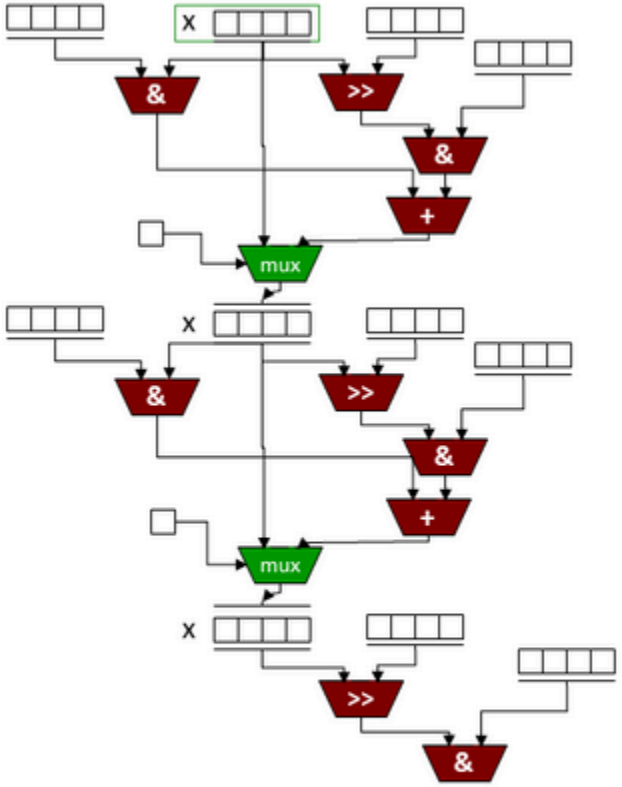
```
int popSketched (bit[W] x)
    implements pop {
    repeat(??) {
        x = (x & ??)
        + ((x >> ??) & ??);
    }
    return x;
}
```



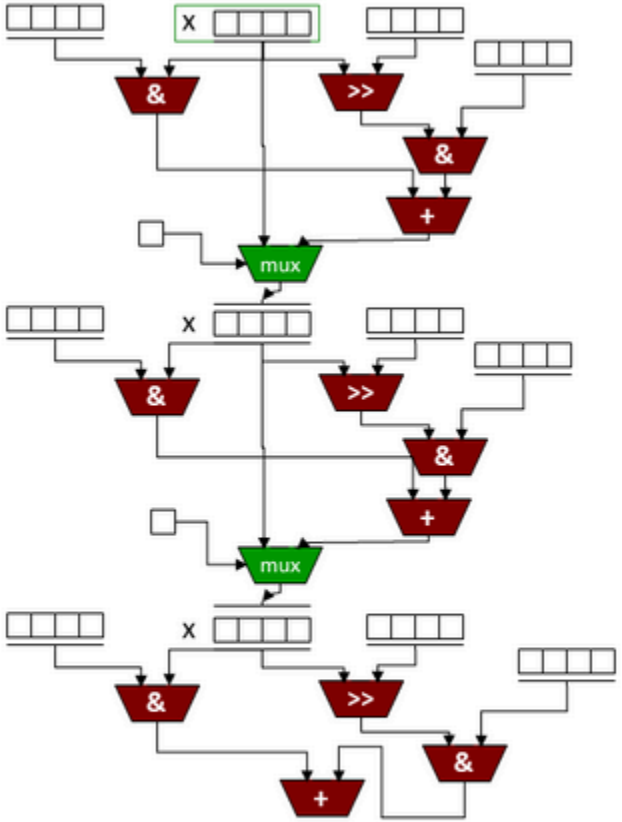
```
int popSketched (bit[W] x)
    implements pop {
    repeat(??) {
        x = (x & ??)
        + ((x >> ??) & ??);
    }
    return x;
}
```



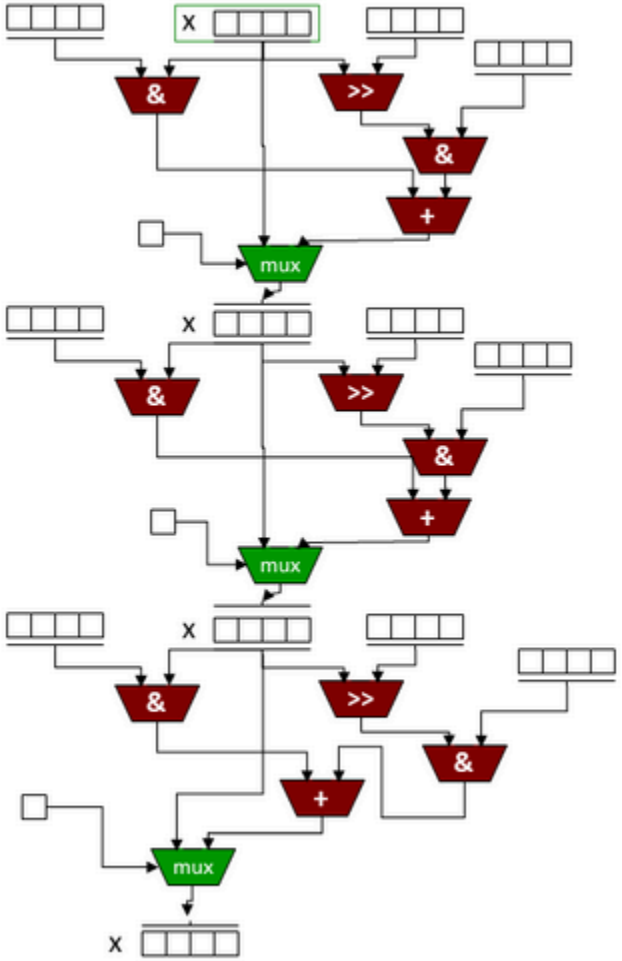
```
int popSketched (bit[W] x)
    implements pop {
    repeat(??) {
        x = (x & ??)
        + ((x >> ??) & ??);
    }
    return x;
}
```



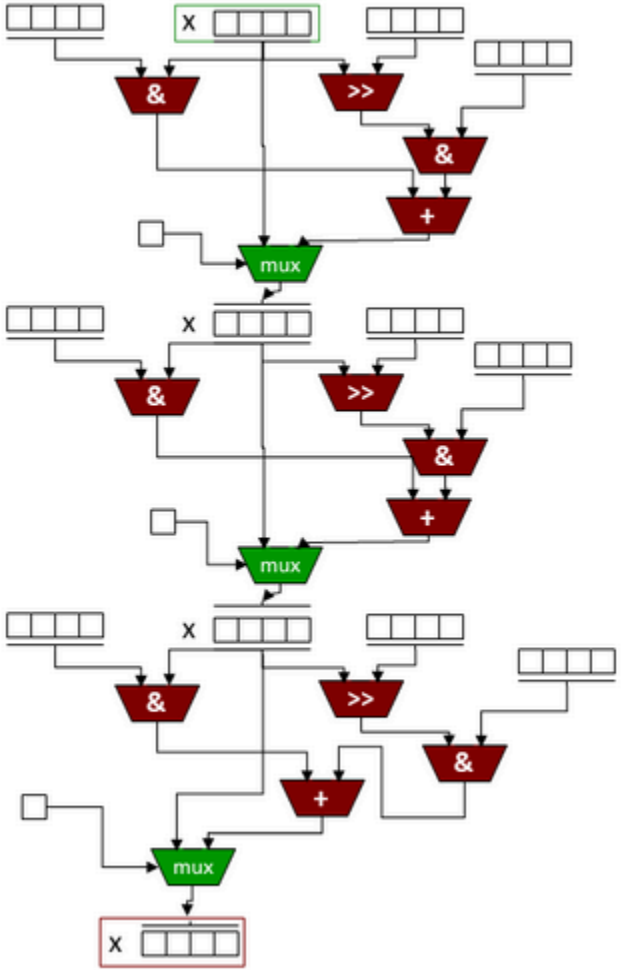
```
int popSketched (bit[W] x)
    implements pop {
    repeat(??) {
        x = (x & ??)
        + ((x >> ??) & ??);
    }
    return x;
}
```



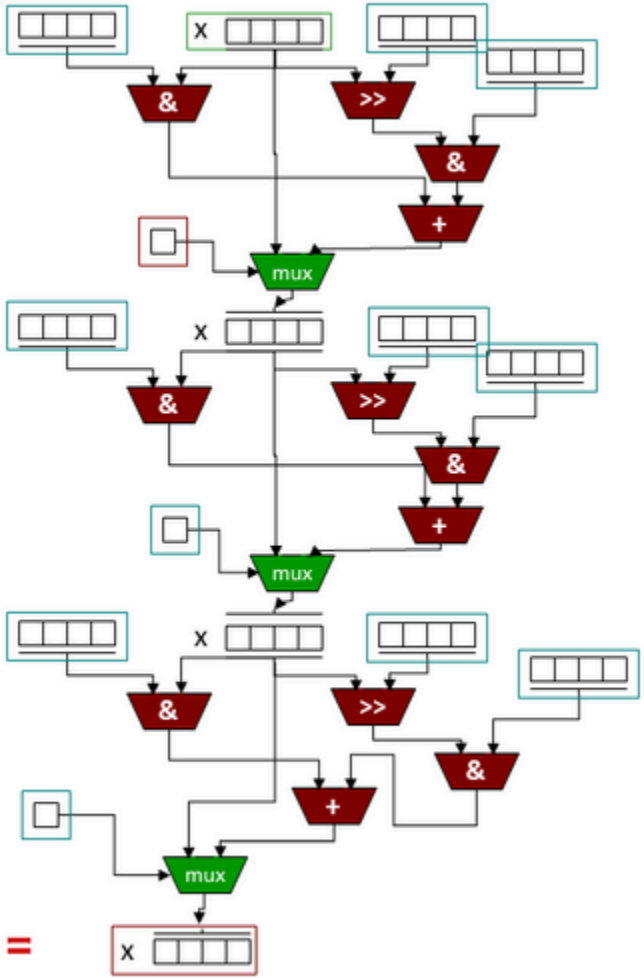
```
int popSketched (bit[W] x)
    implements pop {
    repeat(??) {
        x = (x & ??)
        + ((x >> ??) & ??);
    }
    return x;
}
```



```
int popSketched (bit[W] x)
    implements pop {
    repeat(??) {
        x = (x & ??)
        + ((x >> ??) & ??);
    }
    return x;
}
```



```
int popSketched (bit[W] x)
    implements pop {
    repeat(??) {
        x = (x & ??)
        + ((x >> ??) & ??);
    }
⇒   return x;
}
```



S(x,o) =

Ex : Population count. 0010 0110 → 3

Ex : Population count. 0010 0110 → 3

```
int pop (bit[W] x)
{
   int count = 0;
   for (int i = 0; i < W; i++) {
      if (x[i]) count++;
   }
   return count;
}
```

Ex : Population count. 0010 0110 → 3




```
int pop (bit[W] x)
{
   int count = 0;
   for (int i = 0; i < W; i++) {
      if (x[i]) count++;
   }
   return count;
}
```

Ex : Population count. 0010 0110 → 3




```
int pop (bit[W] x)
{
  int count = 0;
  for (int i = 0; i < W; i++) {
    if (x[i]) count++;
  }
  return count;
}
```

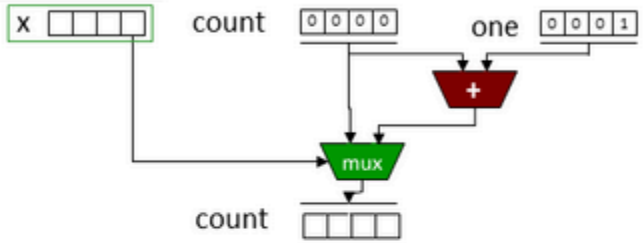
Ex : Population count. 0010 0110 → 3




```
int pop (bit[W] x)
{
   int count = 0;
   for (int i = 0; i < W; i++) {
      if (x[i]) count++;
   }
   return count;
}
```

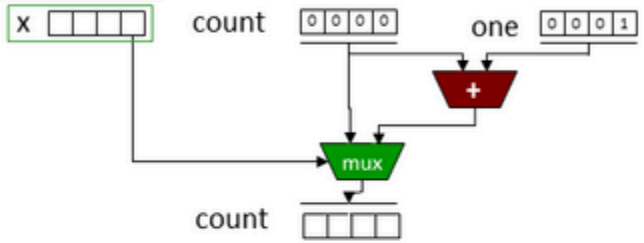
Ex : Population count. 0010 0110 → 3

```
int pop (bit[W] x)
{
   int count = 0;
   for (int i = 0; i < W; i++) {
      if (x[i]) count++;
   }
   return count;
}
```



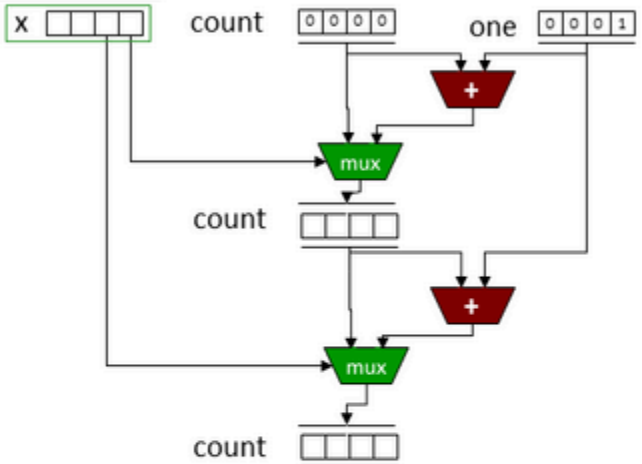
Ex : Population count. 0010 0110 → 3

```
int pop (bit[W] x)
{
   int count = 0;
   for (int i = 0; i < W; i++) {
      if (x[i]) count++;
   }
   return count;
}
```



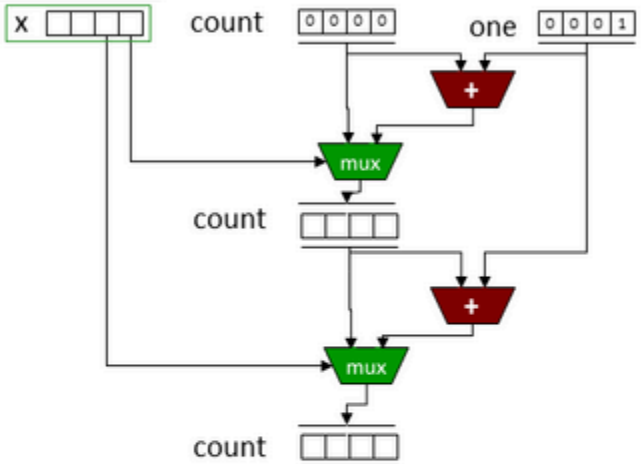
Ex : Population count. 0010 0110 → 3

```
int pop (bit[W] x)
{
   int count = 0;
   for (int i = 0; i < W; i++) {
      if (x[i]) count++;
   }
   return count;
}
```



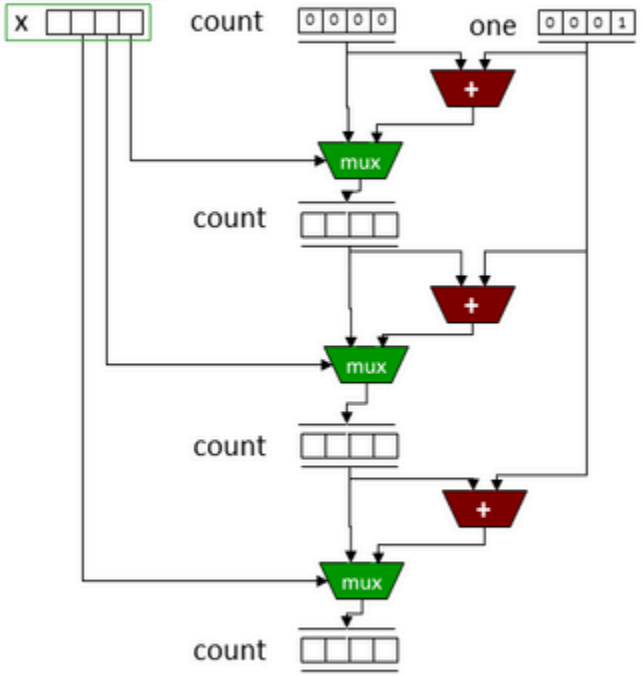
Ex : Population count. 0010 0110 → 3

```
int pop (bit[W] x)
{
   int count = 0;
   for (int i = 0; i < W; i++) {
      if (x[i]) count++;
   }
   return count;
}
```



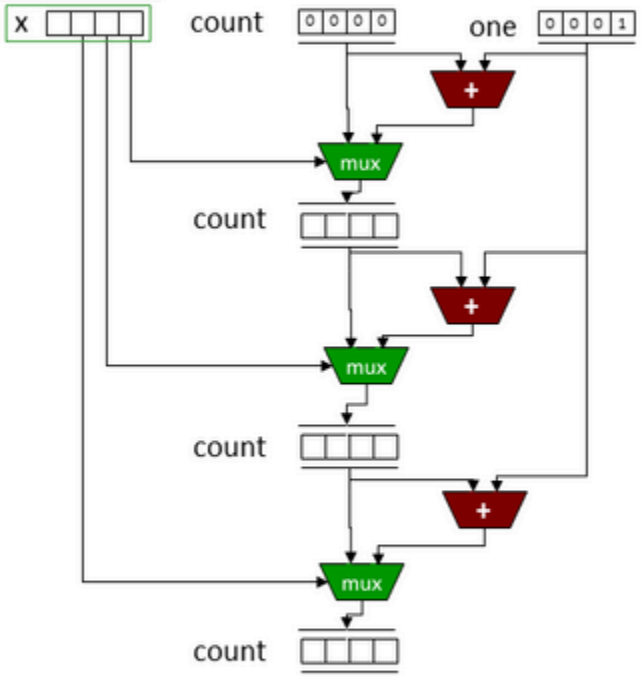
Ex : Population count. 0010 0110 → 3

```
int pop (bit[W] x)
{
   int count = 0;
   for (int i = 0; i < W; i++) {
      if (x[i]) count++;
   }
   return count;
}
```



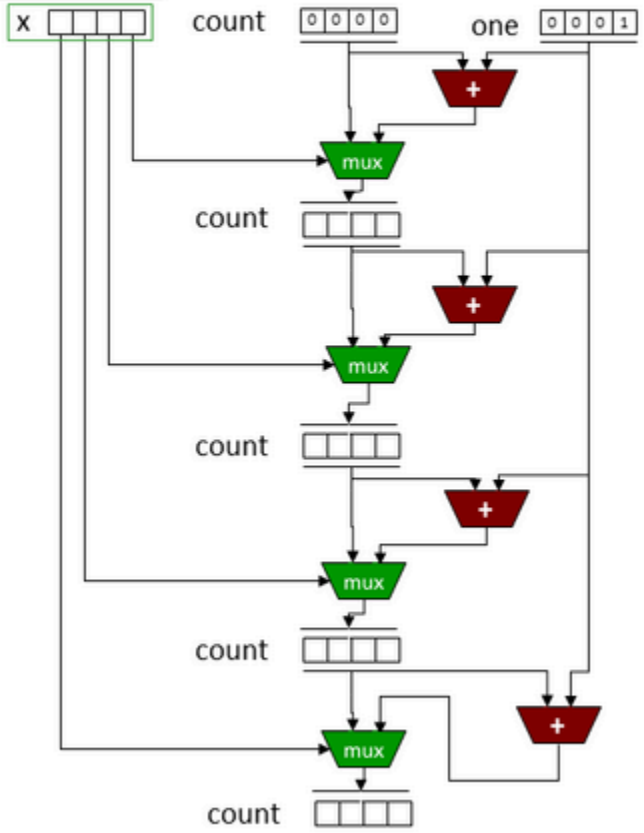
Ex : Population count. 0010 0110 → 3

```
int pop (bit[W] x)
{
   int count = 0;
   for (int i = 0; i < W; i++) {
      if (x[i]) count++;
   }
   return count;
}
```



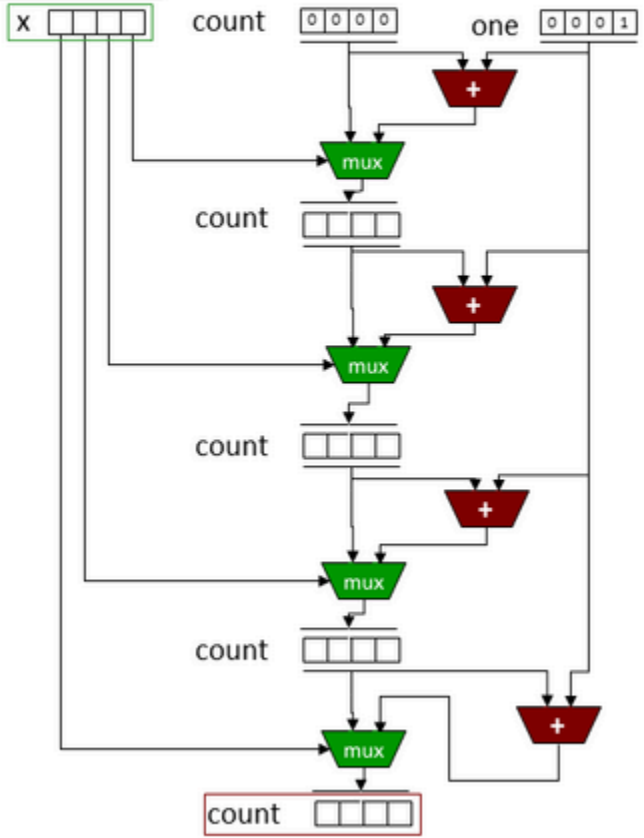
Ex : Population count. 0010 0110 → 3

```
int pop (bit[W] x)
{
   int count = 0;
   for (int i = 0; i < W; i++) {
      if (x[i]) count++;
   }
   return count;
}
```



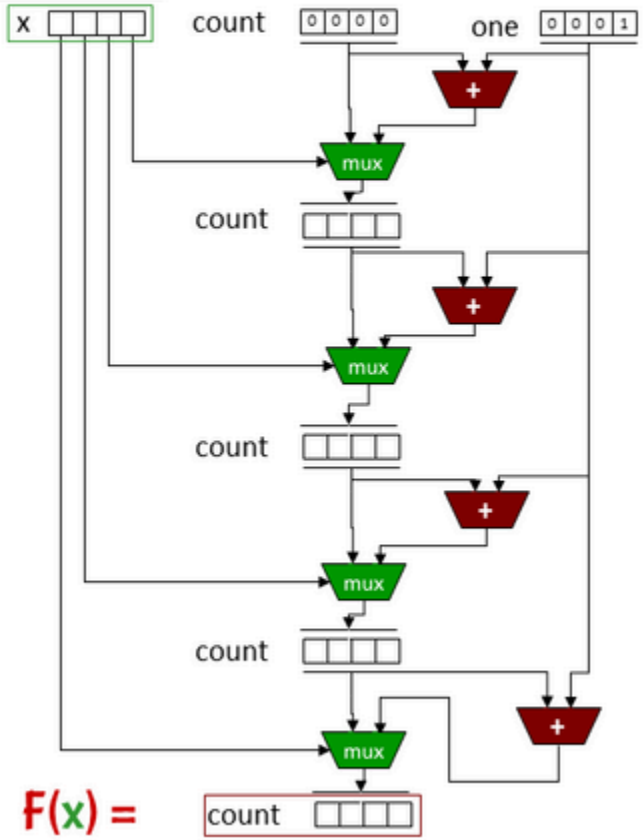
Ex : Population count. 0010 0110 → 3

```
int pop (bit[W] x)
{
   int count = 0;
   for (int i = 0; i < W; i++) {
      if (x[i]) count++;
   }
   return count;
}
```



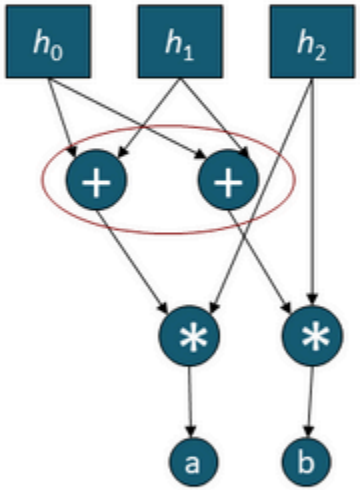
Ex : Population count. 0010 0110 → 3

```
int pop (bit[W] x)
{
   int count = 0;
   for (int i = 0; i < W; i++) {
      if (x[i]) count++;
   }
   return count;
}
```

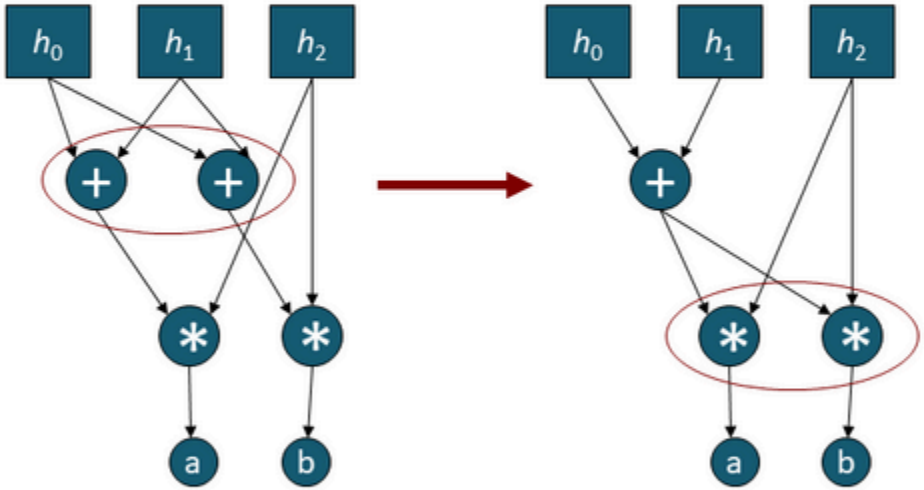


Simplification

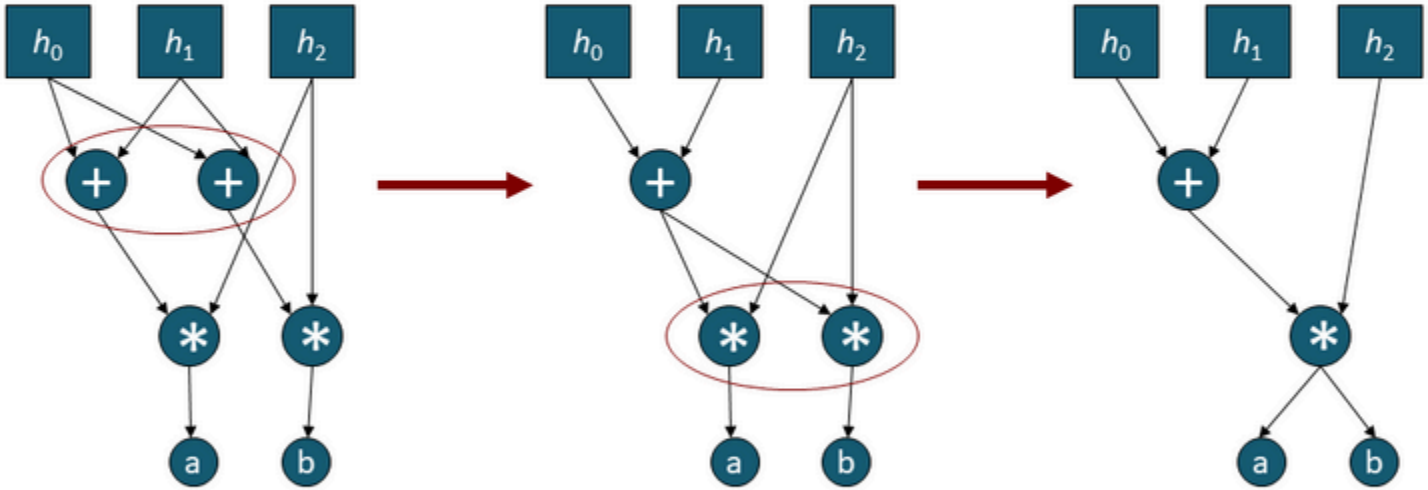
Structural Hashing



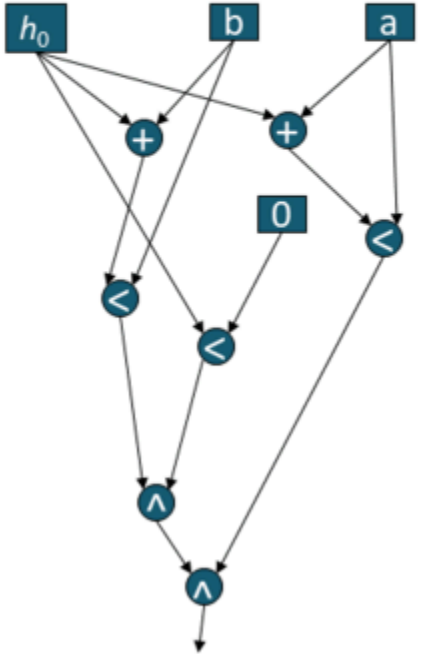
Structural Hashing



Structural Hashing

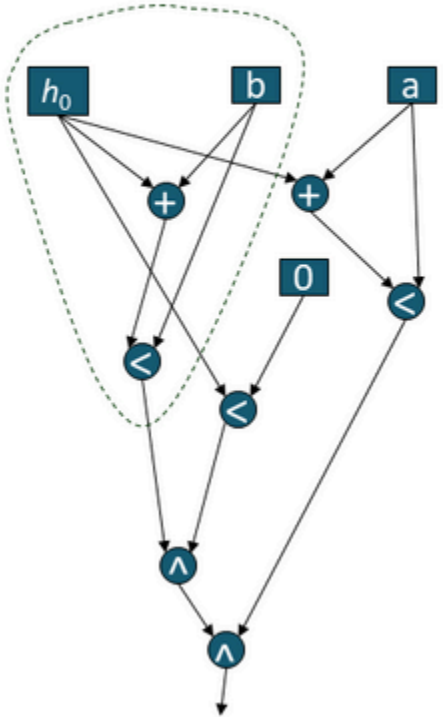


Structural hashing + rewriting



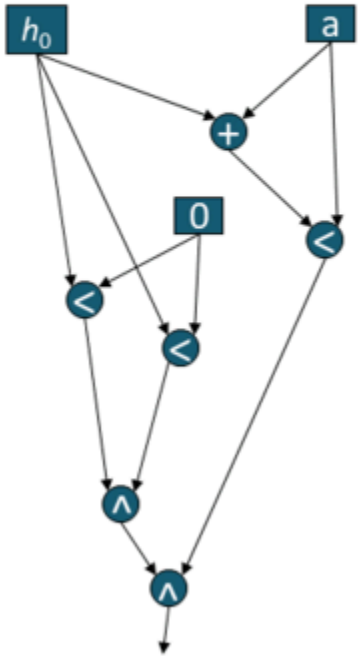
Structural hashing + rewriting

$$X + b < b \rightarrow X < 0$$



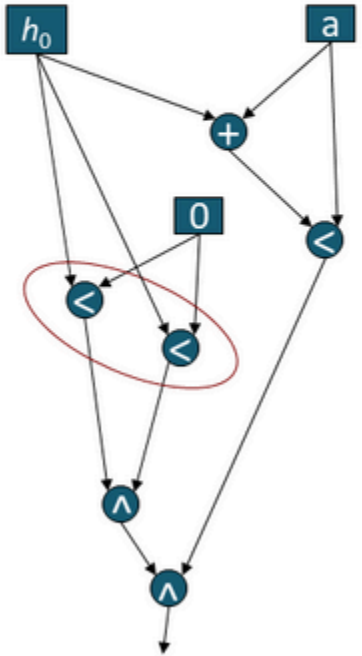
Structural hashing + rewriting

$$X + b < b \rightarrow X < 0$$



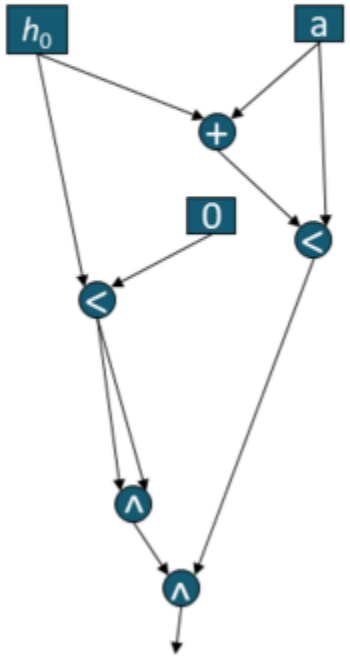
Structural hashing + rewriting

$X + b < b \rightarrow X < 0$



Structural hashing + rewriting

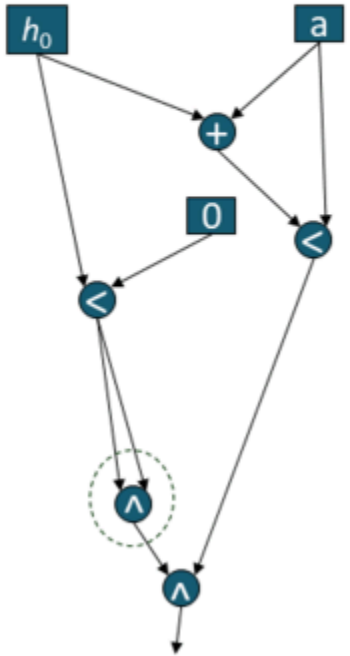
$$X + b < b \rightarrow X < 0$$



Structural hashing + rewriting

$X + b < b \rightarrow X < 0$

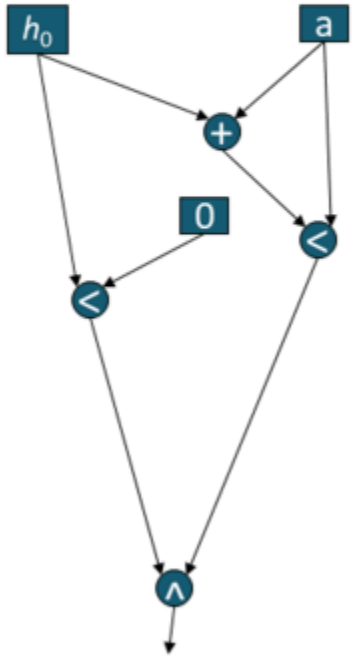
$X \wedge X \rightarrow X$



Structural hashing + rewriting

$X + b < b \rightarrow X < 0$

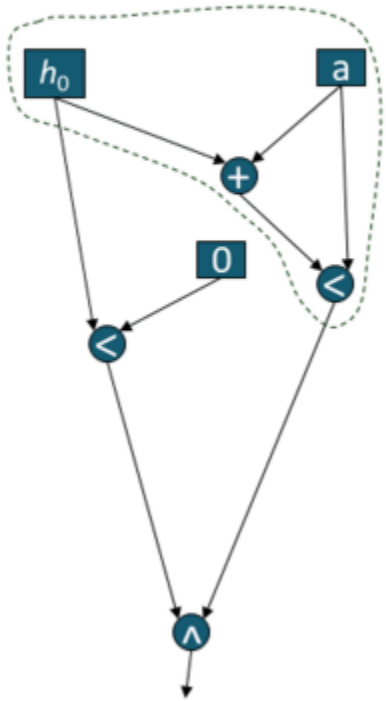
$X \wedge X \rightarrow X$



Structural hashing + rewriting

$X + b < b \rightarrow X < 0$

$X \wedge X \rightarrow X$



Structural hashing + rewriting

$X + b < b \rightarrow X < 0$

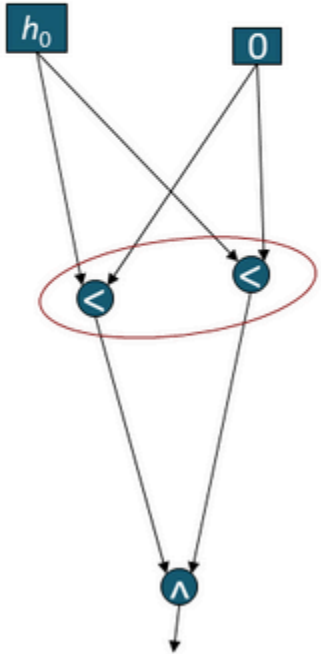
$X \wedge X \rightarrow X$



Structural hashing + rewriting

$X + b < b \rightarrow X < 0$

$X \wedge X \rightarrow X$



Structural hashing + rewriting

$X + b < b \rightarrow X < 0$

$X \wedge X \rightarrow X$



Structural hashing + rewriting

$X + b < b \rightarrow X < 0$

$X \wedge X \rightarrow X$



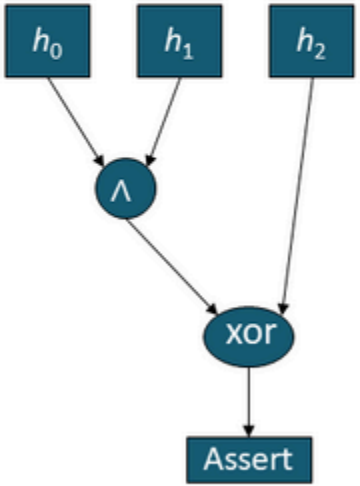
Structural hashing + rewriting

$X + b < b \rightarrow X < 0$

$X \wedge X \rightarrow X$



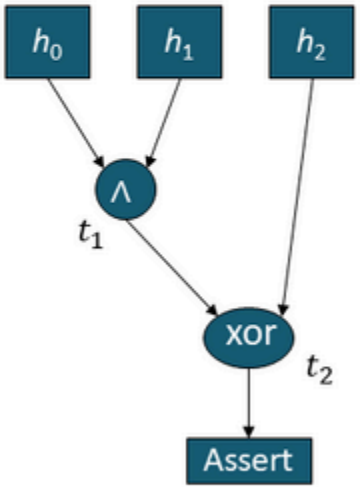
Constraints to SAT



SAT Solver only understands CNF

- Sum (OR) of variables and their negation
- Equivalent to $\bigwedge_{i \in X} l_i \Rightarrow l_j$

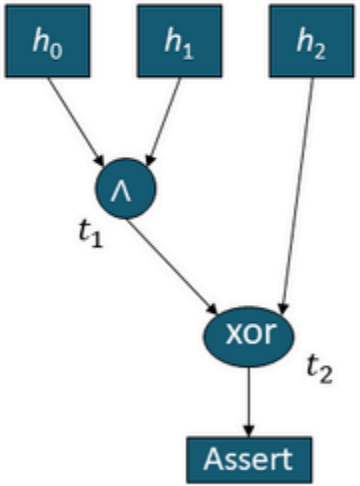
Constraints to SAT



SAT Solver only understands CNF

- Sum (OR) of variables and their negation
- Equivalent to $\Lambda_{i \in X} l_i \Rightarrow l_j$

Constraints to SAT

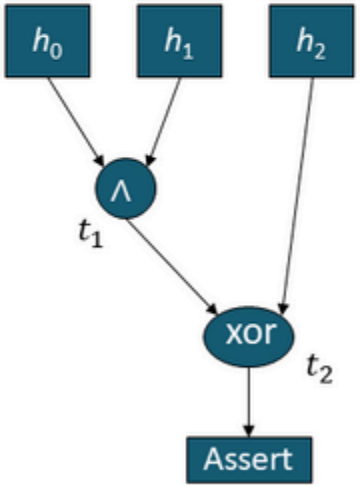


$h_0 \wedge h_1 \Rightarrow t_1$
$t_1 \Rightarrow h_0$
$t_1 \Rightarrow h_1$

SAT Solver only understands CNF

- Sum (OR) of variables and their negation
- Equivalent to $\bigwedge_{i \in X} l_i \Rightarrow l_j$

Constraints to SAT



$h_0 \wedge h_1 \Rightarrow t_1$
$t_1 \Rightarrow h_0$
$t_1 \Rightarrow h_1$

$t_1 \wedge h_2 \Rightarrow \overline{t_2}$
$\overline{t_1} \wedge \overline{h_2} \Rightarrow \overline{t_2}$
$t_1 \wedge \overline{h_2} \Rightarrow t_2$
$\overline{t_1} \wedge h_2 \Rightarrow t_2$

SAT Solver only understands CNF

- Sum (OR) of variables and their negation
- Equivalent to $\bigwedge_{i \in X} l_i \Rightarrow l_j$

SAT Solver

SAT solving in a nutshell

Constraint database



SAT Solver

SAT solving in a nutshell

Constraint database



Constraint is a CNF Clause

$x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5 \vee x_6$



SAT Solver

SAT solving in a nutshell

Constraint database



x_1

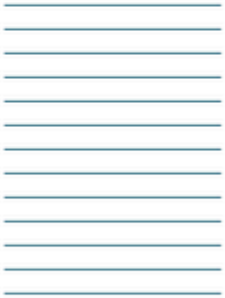
Constraint is a CNF Clause

$x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5 \vee x_6$

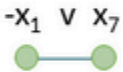
SAT Solver

SAT solving in a nutshell

Constraint database



x_1



x_7

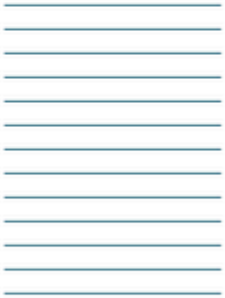
Constraint is a CNF Clause



SAT Solver

SAT solving in a nutshell

Constraint database



x_1

$-x_1 \vee x_7$



x_7

x_5

Constraint is a CNF Clause

$x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5 \vee x_6$

SAT Solver

SAT solving in a nutshell

Constraint database



x_1

$-x_1 \vee x_7$



x_5

x_7

$-x_7 \vee x_6 \vee -x_5$



x_6

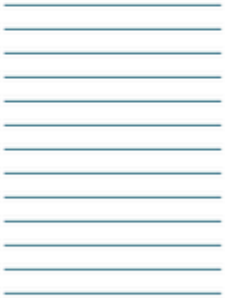
Constraint is a CNF Clause

$x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5 \vee x_6$

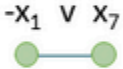
SAT Solver

SAT solving in a nutshell

Constraint database



x_1



x_7



x_6

x_5

$-x_9$

Constraint is a CNF Clause



SAT Solver

SAT solving in a nutshell

Constraint database



x_1

$-x_1 \vee x_7$

x_5

x_7

$-x_9$

$-x_7 \vee x_6 \vee -x_5$

$-x_5 \vee -x_3 \vee x_9$

x_6

$-x_3$

Constraint is a CNF Clause

$x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5 \vee x_6$

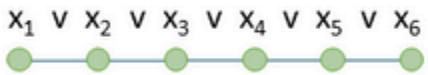
SAT Solver

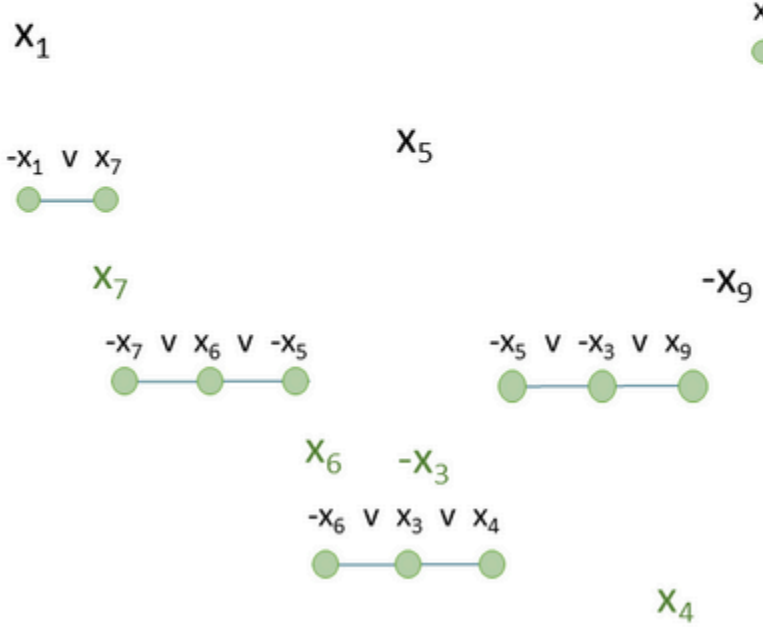
SAT solving in a nutshell

Constraint database



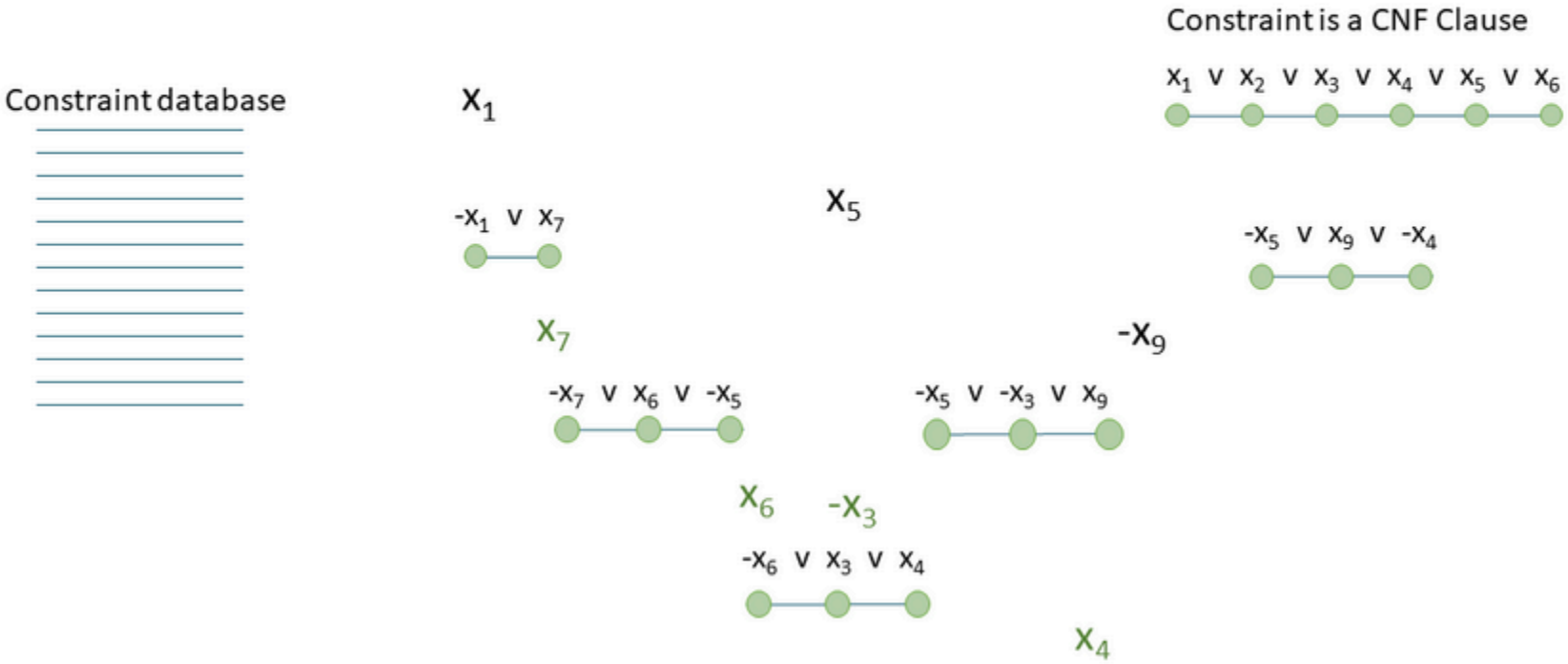
Constraint is a CNF Clause





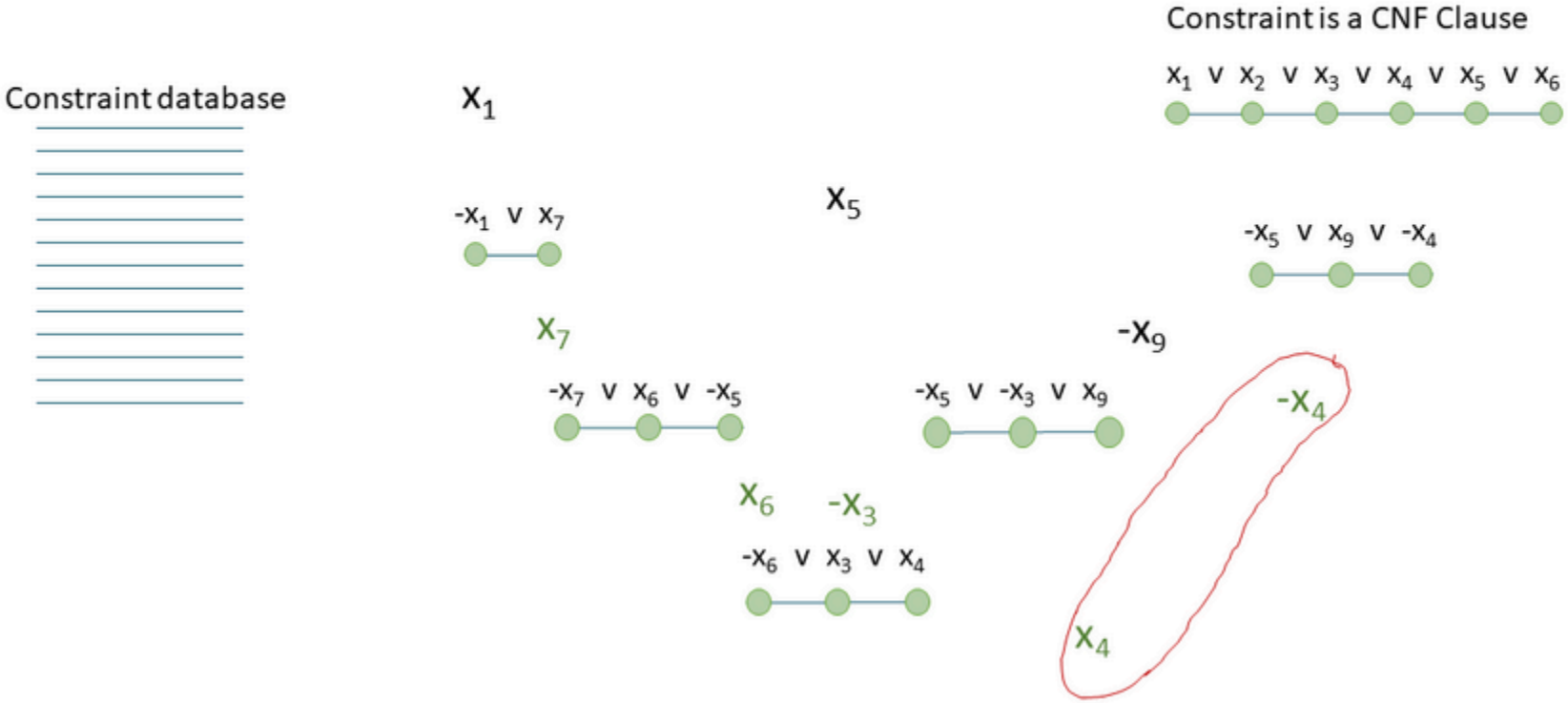
SAT Solver

SAT solving in a nutshell



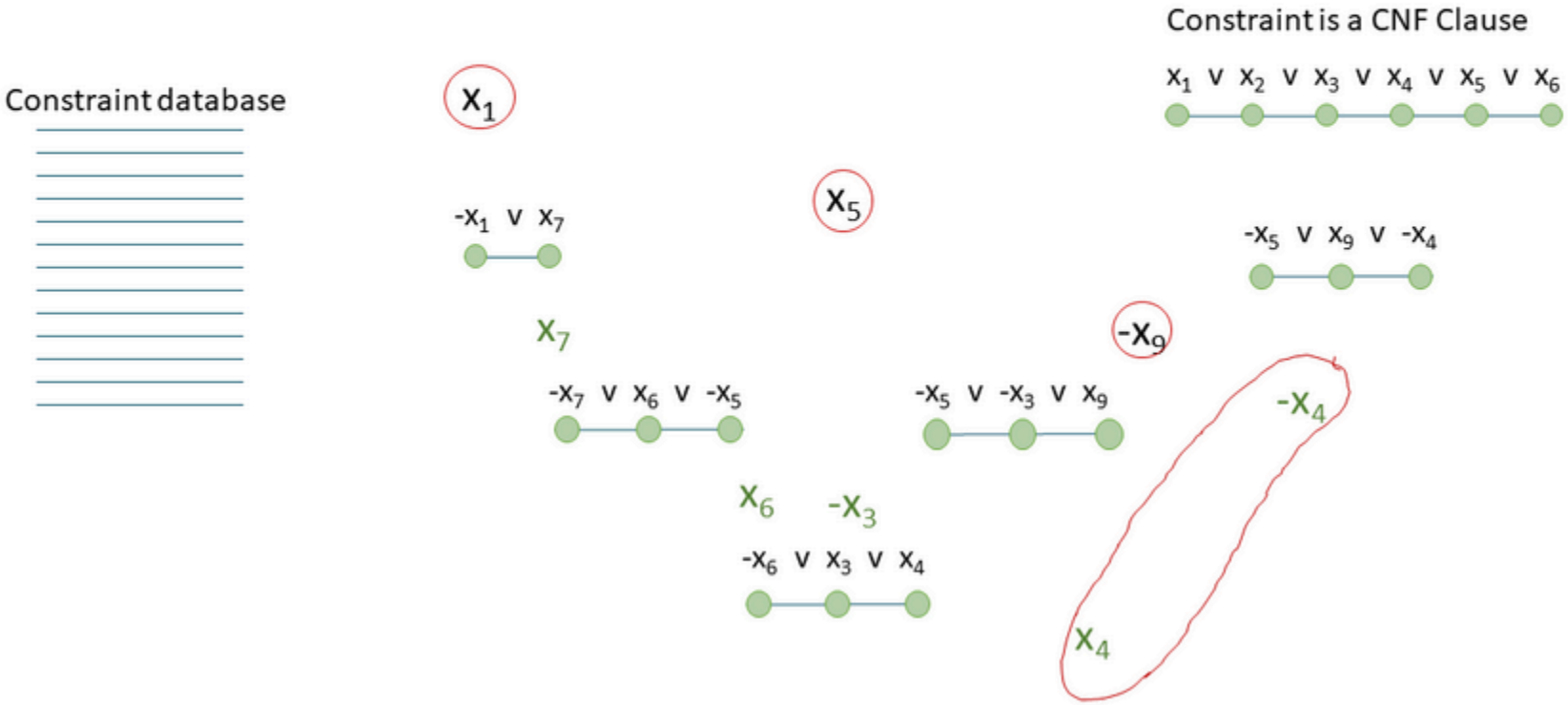
SAT Solver

SAT solving in a nutshell



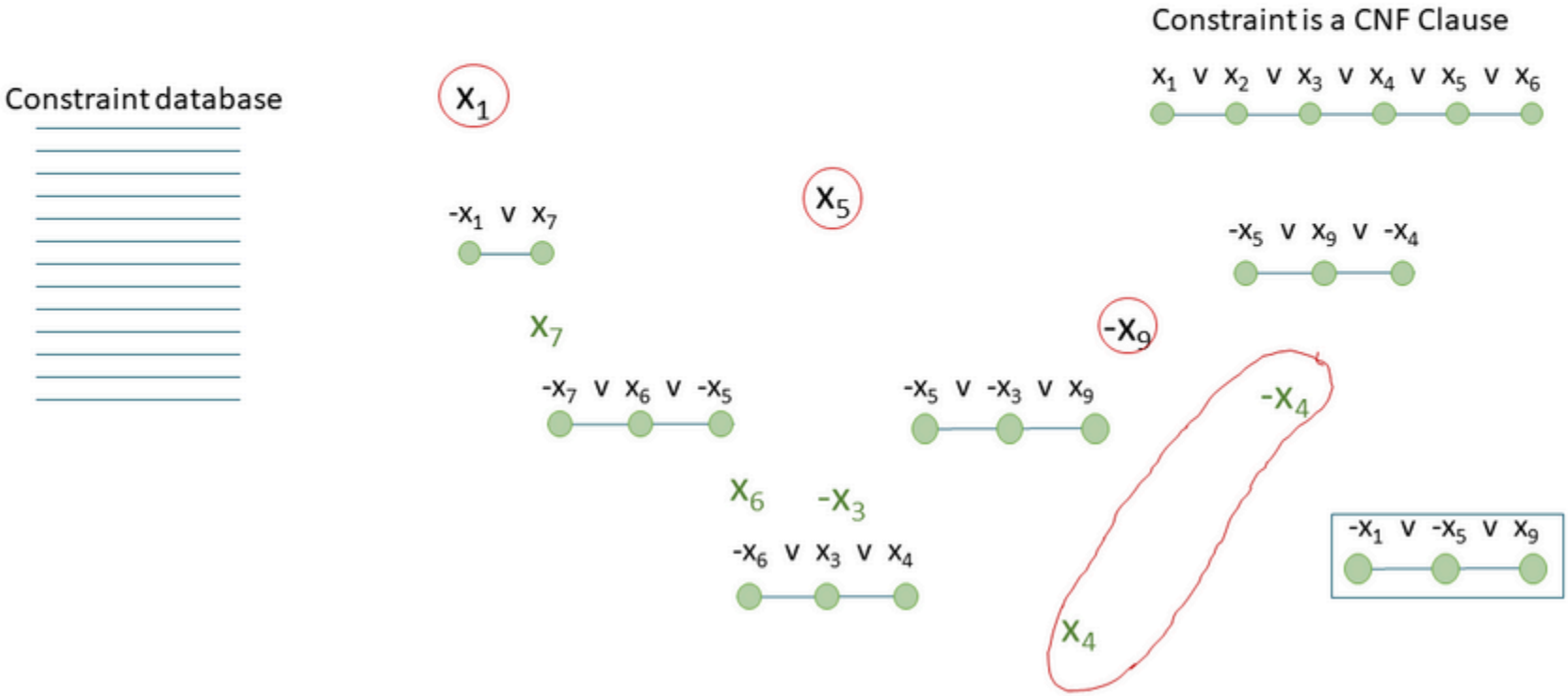
SAT Solver

SAT solving in a nutshell



SAT Solver

SAT solving in a nutshell



What about Arithmetic?

1) Bit-blast

2) Unary encoding

3) SMT